

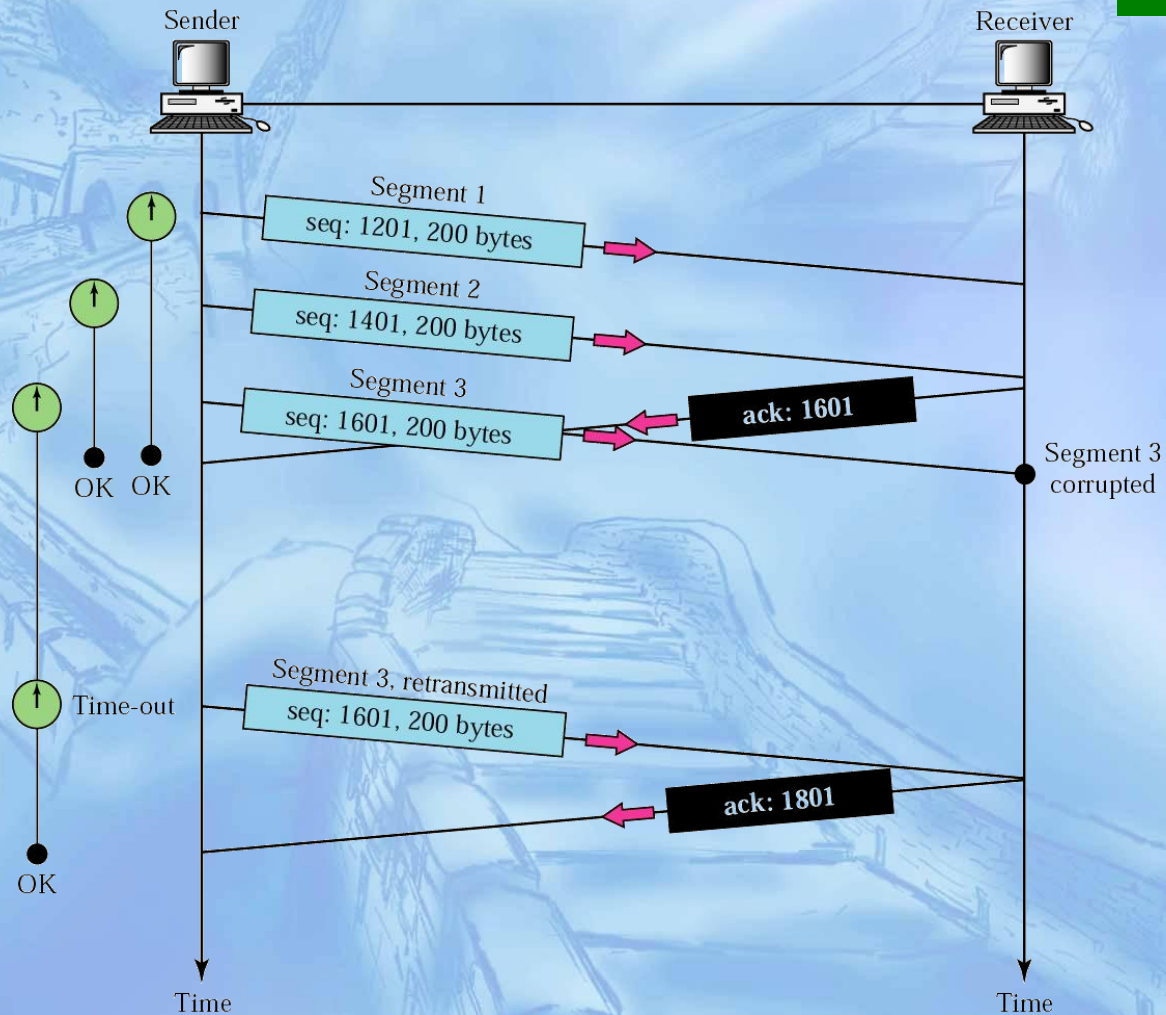


差错控制

TCP使用差错控制提供可靠性。差错控制的方法为：用报文段中的**校验和**字段进行检测差错是否发生，如果发生，丢弃该报文段，让发送端**超时重发**；如果没有发生，向发送端发送**确认**进行证实。

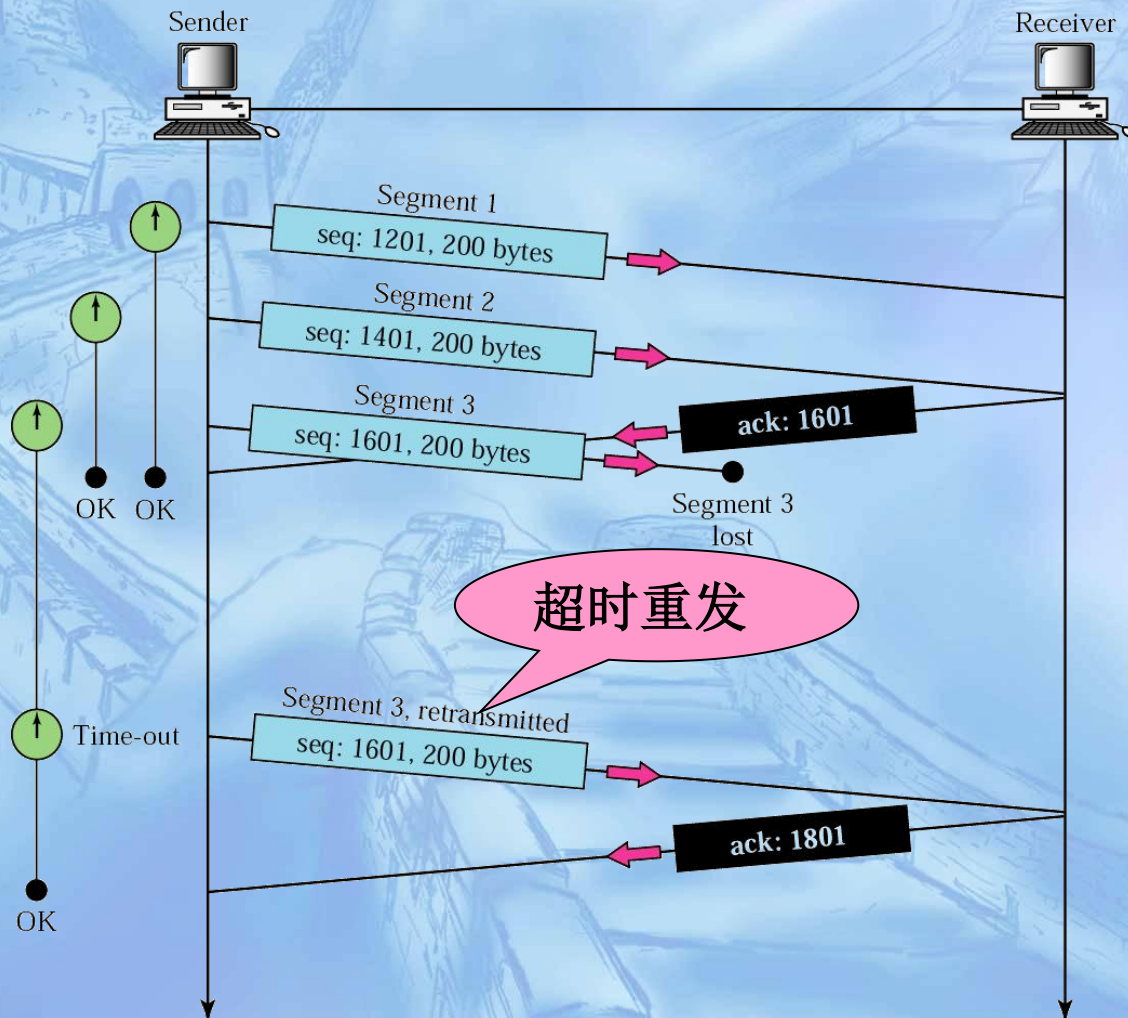


例:受损伤的报文段



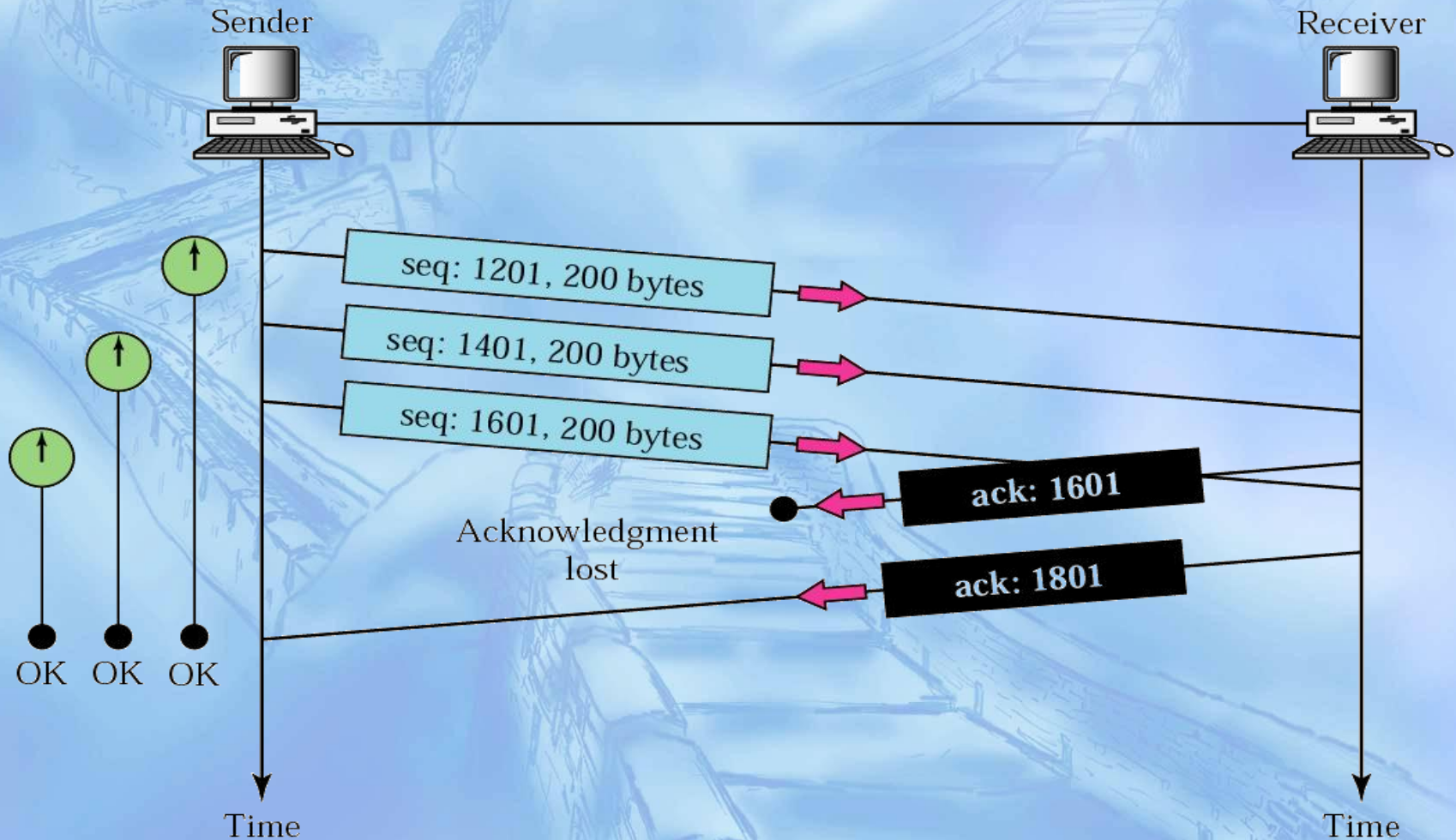


例:丢失的报文段





例:丢失的确认





例:其他情况

重复的报文段

处理方法：目的TCP丢弃重复的报文段

。

失序的报文段

处理方法：目的TCP对失序的报文段不确认，直到收到所有它以前的报文段为止。



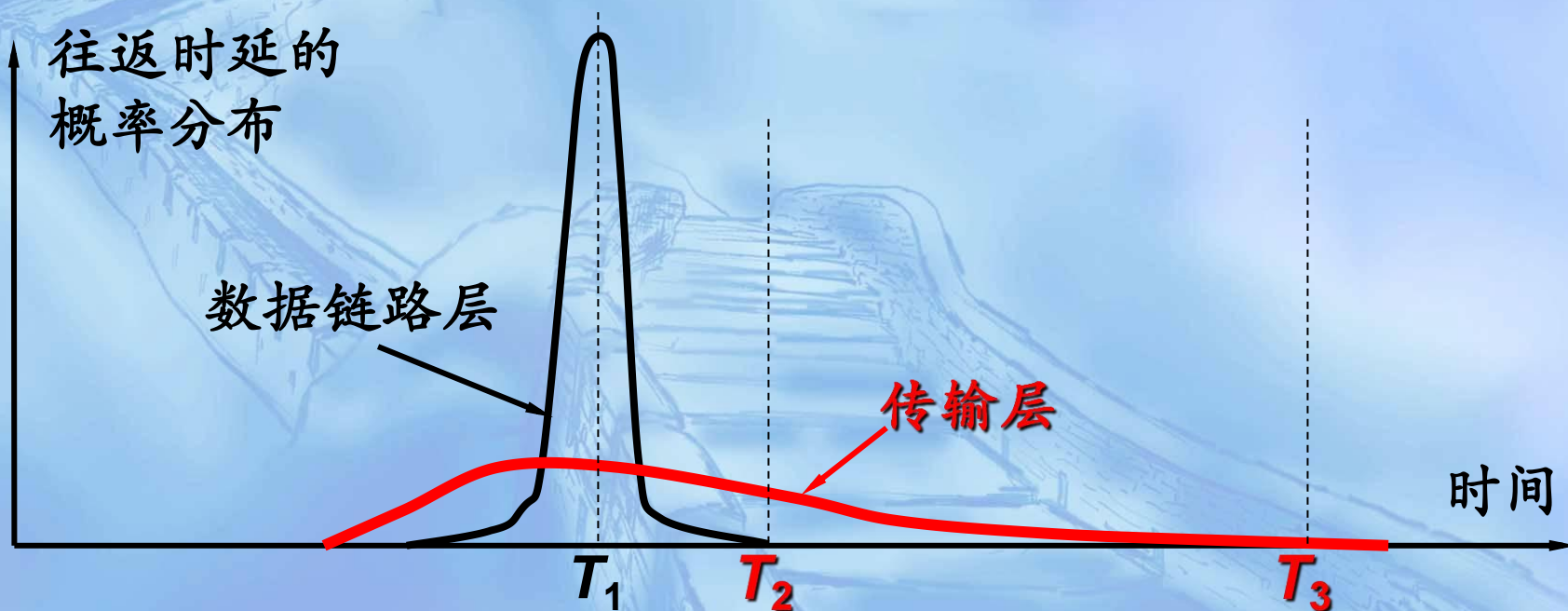
重传计时器

TCP 每发送一个报文段，就对这个报文段设置一次计时器。如果在计时器截止时间到之前收到对此特定报文段的确认，则撤消此计时器，否则就要重传此报文段，并将计时器复位。



如何确定重传时间？

由于TCP的下层是一个互连网环境，IP数据报所选择的路由变化很大。因而传输层的往返时延（RTT）的方差也很大。





往返时延的自适应算法

- ✎ TCP保留了 RTT 的一个**加权平均往返时间 SRTT**（这又称为**平滑的往返时间**）；
- ✎ 第一次测量到 RTT 样本时，SRTT 值就取为所测量到的 RTT 样本值。以后每测量到一个新的 RTT 样本，就按下式重新计算一次 SRTT：

$$\text{SRTT} = \alpha \times \text{SRTT} + (1 - \alpha) \times \text{新的 RTT 样本}$$

式中， $0 \leq \alpha < 1$ 。若 α 很接近于零，表示 RTT 值更新较快。若选择 α 接近于 1，则表示 RTT 值更新较慢。

- ✎ **RFC 6298** 推荐的 α 值为 **7/8**。



超时重传时间 (RetransmissionTime-Out)

- ✗ RTO应略大于上面得出的加权平均往返时间 SRTT;
- ✗ RFC 6298 建议使用下式计算 RTO:

$$\text{RTO} = \text{SRTT} + 4 \times \text{RTTVAR}$$

RTTVAR是RTT的偏差的加权平均值。

- ✗ RFC 6298 建议这样计算 RTTVAR。第一次测量时, RTTVAR值取为测量到的 RTT 样本值的一半。在以后的测量中, 则使用下式计算加权平均的 RTTVAR:

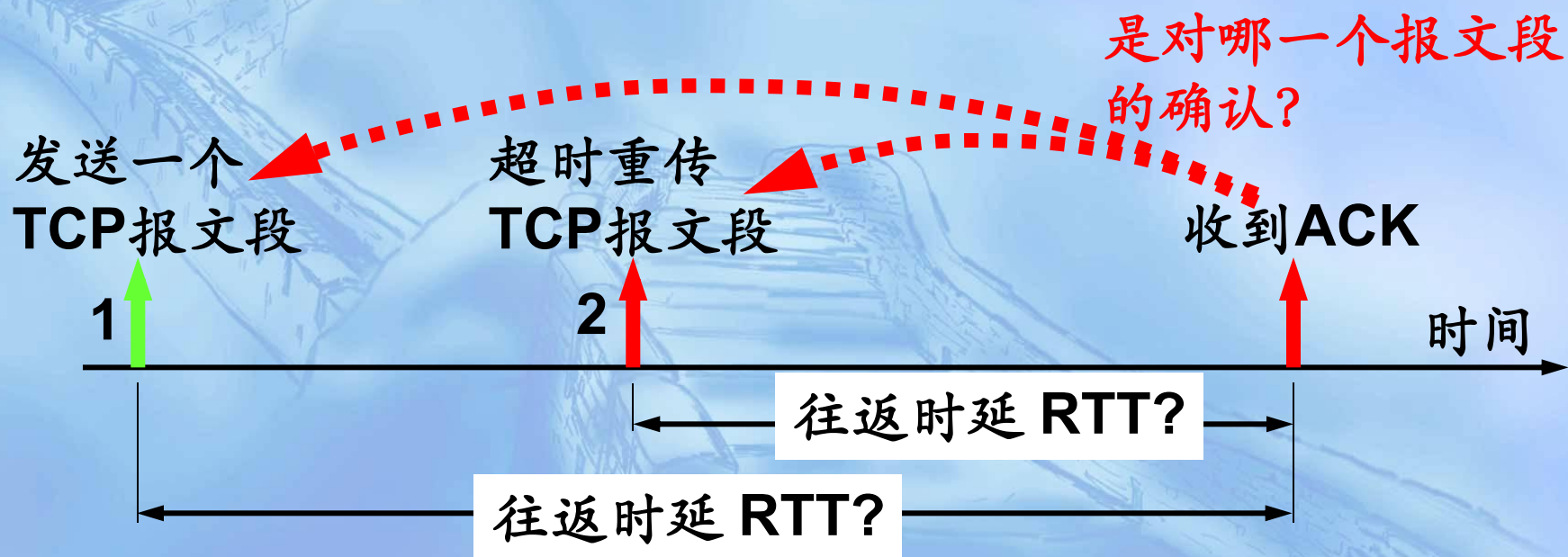
$$\text{RTTVAR} = \beta \times \text{RTTVAR} + (1 - \beta) \times |\text{SRTT} - \text{新的RTT样本}|$$

β 是个小于 1 的系数, 其推荐值是 3/4。



计算RTT中的问题

TCP 报文段 1 没有收到确认。重传（即报文段 2）后，收到了确认报文段 **ACK**。如何判定此确认报文段是对原来的报文段 1 的确认，还是对重传的报文段 2 的确认？





解决:Karn算法

✍ 算法

在计算平均往返时延RTT时，只要报文段重传了，就不采用其往返时延样本。这样得出的平均往返时延RTT和重传时间就较准确。

✍ 问题

设想出现这样的情况：报文段的时延突然增大了很多。因此在原来得出的重传时间内，不会收到确认报文段，于是重传报文段。但根据Karn算法，不考虑重传报文段的RTT样本。这样，重传时间就无法更新。



解决:修正的Karn算法

报文段每重传一次，就将重传时间增大一些：

新的重传时间 = $\gamma \times$ (旧的重传时间)

系数 γ 的典型值是2。当不再发生报文段的重传时，才根据报文段的往返时延更新平均往返时延 **RTT** 和重传时间的数值。

实践证明，这种策略较为合理。



流量控制

✍ 问题

发送方发送能力大于接收方接收能力，会导致接收方来不及接收。

✍ 解决

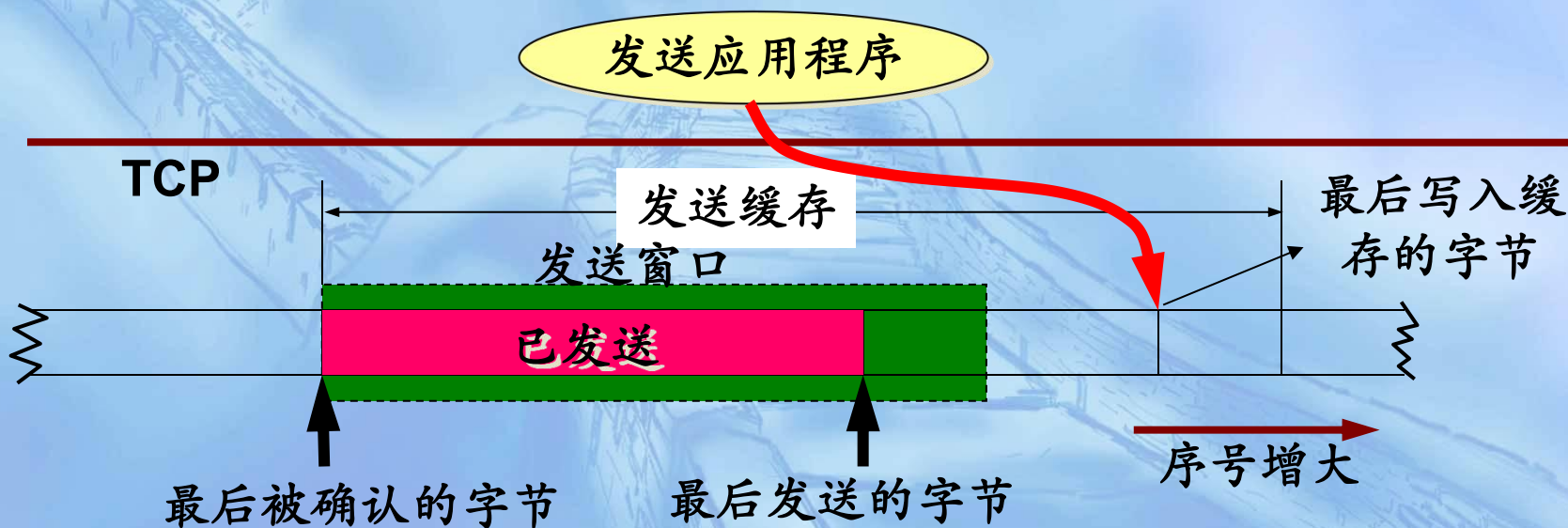
控制发送方的速率，使其不超过接收方所能承受的能力，常采用滑动窗口机制进行控制。



滑动窗口

✎ 发送窗口

每个发送端设置一个，用于表示发送端可以发送数据的大小。发送窗口的大小在连接建立时由双方商定，在数据传输过程中取决于接收端。

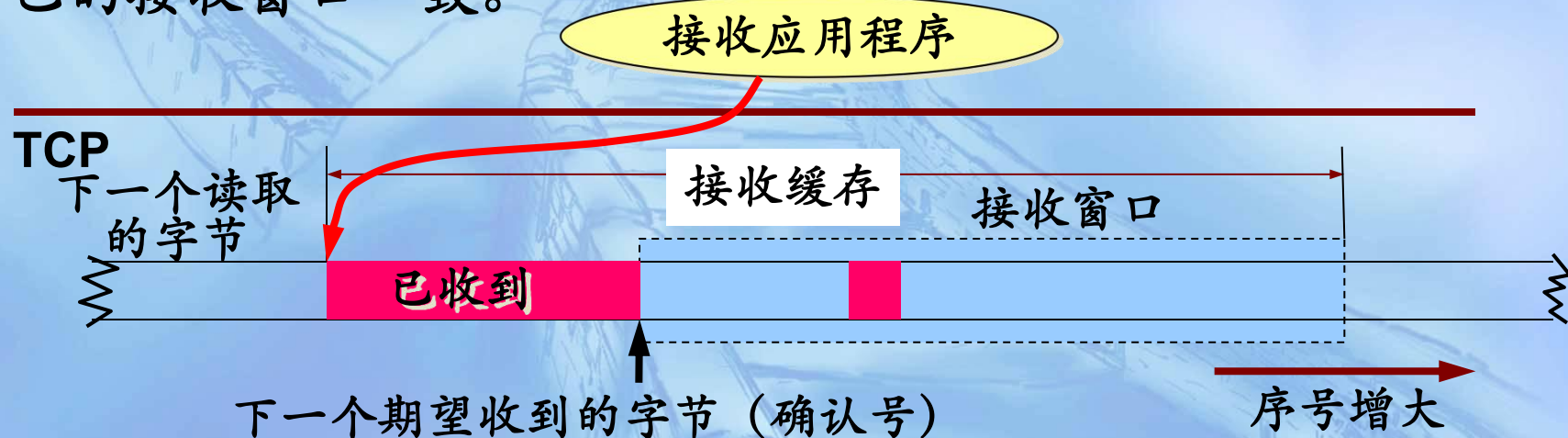




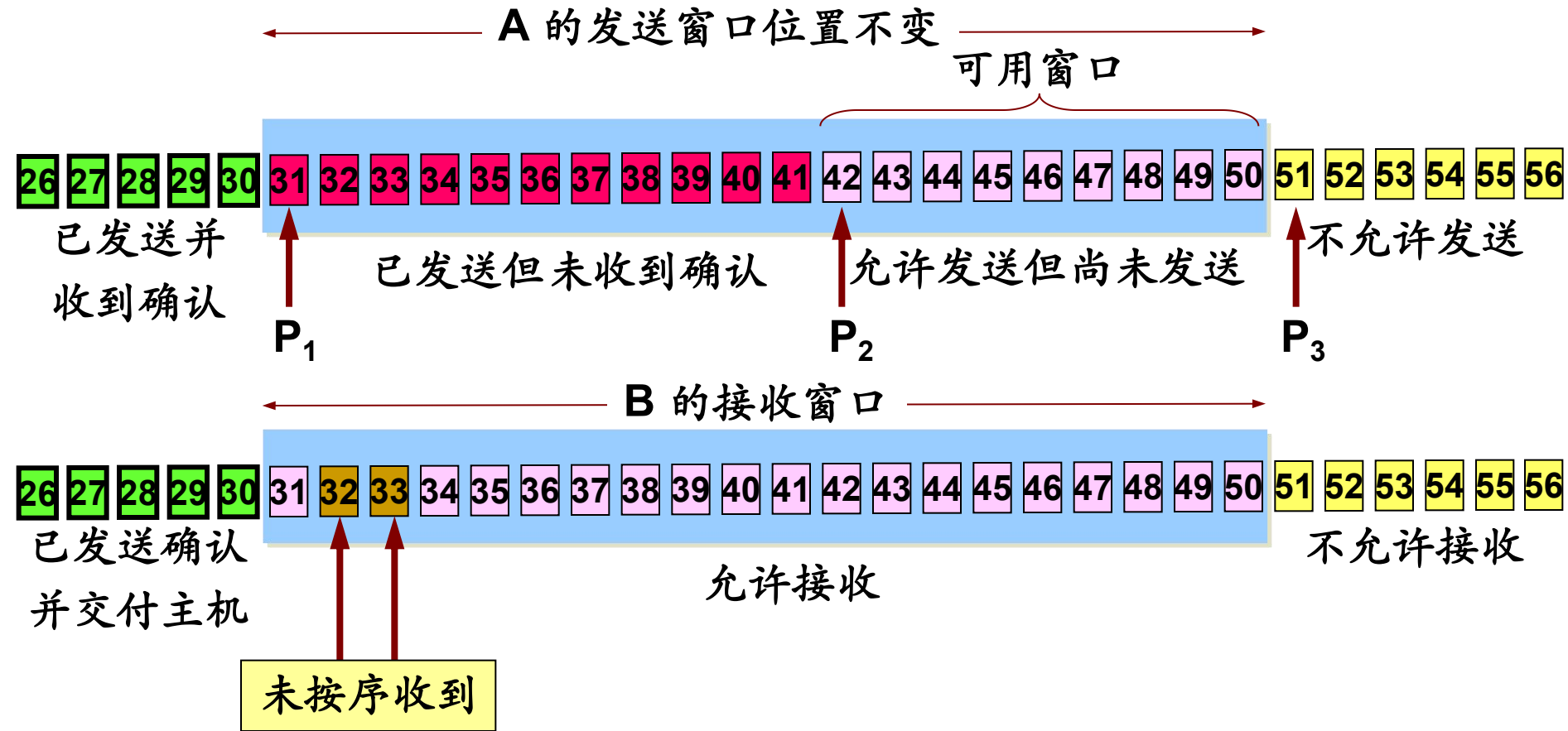
滑动窗口

接收窗口

每个接收端设置一个，用于表示接收端允许接收的数据的大小。接收窗口的大小在连接建立时由双方商定，在数据传输过程中，接收端可根据自己的资源情况，动态调整接收窗口的大小，然后告诉对方，使对方的发送窗口与自己的接收窗口一致。



A 发送了 11 个字节的数据



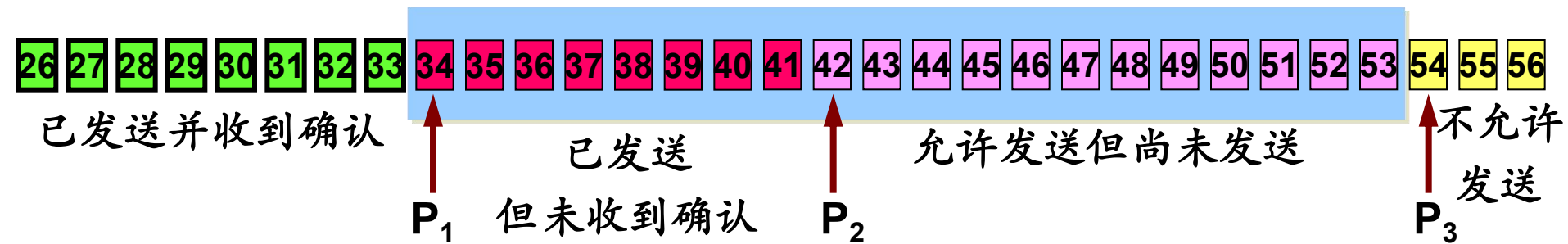
$P_3 - P_1 = A$ 的发送窗口（又称为**通知窗口**）

$P_2 - P_1 =$ 已发送但尚未收到确认的字节数

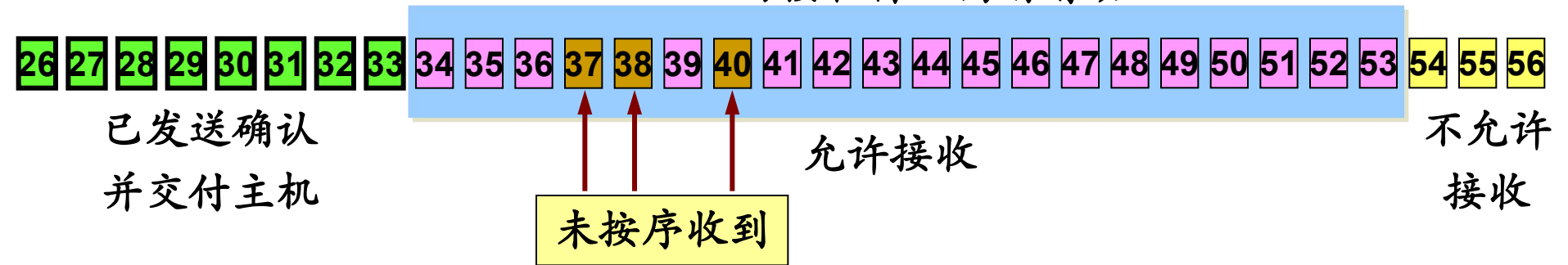
$P_3 - P_2 =$ 允许发送但尚未发送的字节数（又称为**可用窗口**）

A 收到新的确认号，发送窗口向前滑动

A 的发送窗口向前滑动 →



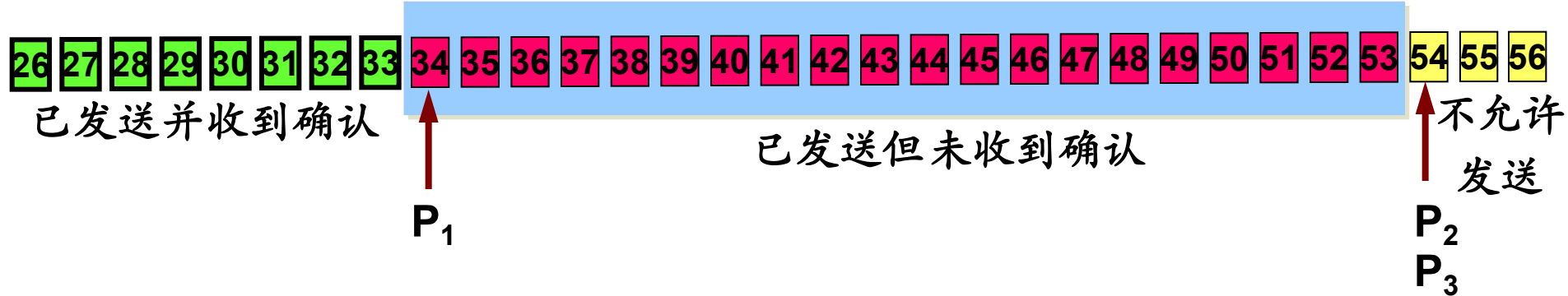
B 的接收窗口向前滑动 →



先存下，等待缺少的
数据的到达

**A 的发送窗口内的序号都已用完，
但还没有再收到确认，必须停止发送。**

A 的发送窗口已满，有效窗口为零



例:利用可变窗口大小进行流量控制

(TCP报文段长100B, 双方确定的最初窗口值是 400B)





滑动窗口的本质

上述滑动窗口的本质是通过**两个缓存**和**一个窗口**来实现的。发送端设置一个缓存，用于存储从应用程序来的数据，对这个缓存设置一个窗口，只要此窗口大小不是0就可以发送数据。接收端也设置一个缓存，用于存储刚接收到的数据，以便应用程序取走，接收端会将自身缓存剩余情况告知发送端，以便发送端改变窗口大小。



糊涂窗口综合症

当发送应用程序产生数据很慢，或接收应用程序吸收数据很慢，会使发送数据的报文段很小，引起操作效率的降低。



发送端

✎ 问题现象

例如：例如一次产生1B，会产生只包括1B数据的TCP报文段（40B的开销：20B的TCP首部和20B的IP首部），结果有很多41B的TCP报文段在Internet中传输。

✎ 解决方法

采用**Nagle算法**：① 发送端将从应用程序收到的第一块数据发送出去，哪怕只有一个字节。② 将后面到达的数据都缓存起来，当收到对方的确认或数据已积累到可以装成一个最大的报文段时，将缓存中的所有数据组装成一个报文段进行发送。③ 对剩余的传输，重复上一步骤。



接收端

✍ 问题现象

例如：一次吸收1B，也会使发送端产生只包括1B的TCP报文段（40B的开销：20B的TCP首部和20B的IP首部），结果有很多41B的TCP报文段在Internet中传输。

✍ 解决方法

可采用两个方法解决：① **Clark解决方法**：只要有数据到达就发送确认，但宣布的窗口大小为0，直至缓存空间已能放入最大长度的报文段，或缓存空间的一半已经空了。② **延迟的确认**：当一个报文段到达时并不立即发送确认，直到缓存有足够的空间为止。